

COMP2522

Lesson 4: interfaces

Review

- private or?

Consider whether it should be overridable

- final or?

Consider whether it should be overridable or extended or changed

- abstract or?

- static or?

- class or?

INTERFACE

Interfaces We Have Seen

- List
- Iterator
- <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/List.html>

Declare at the Level of the Interface

- `final List<String> notes;      // NOT ArrayList`
- `notes = new ArrayList<>();    // or LinkedList, etc....`
- Substitute a List-implementer class

Heads Up

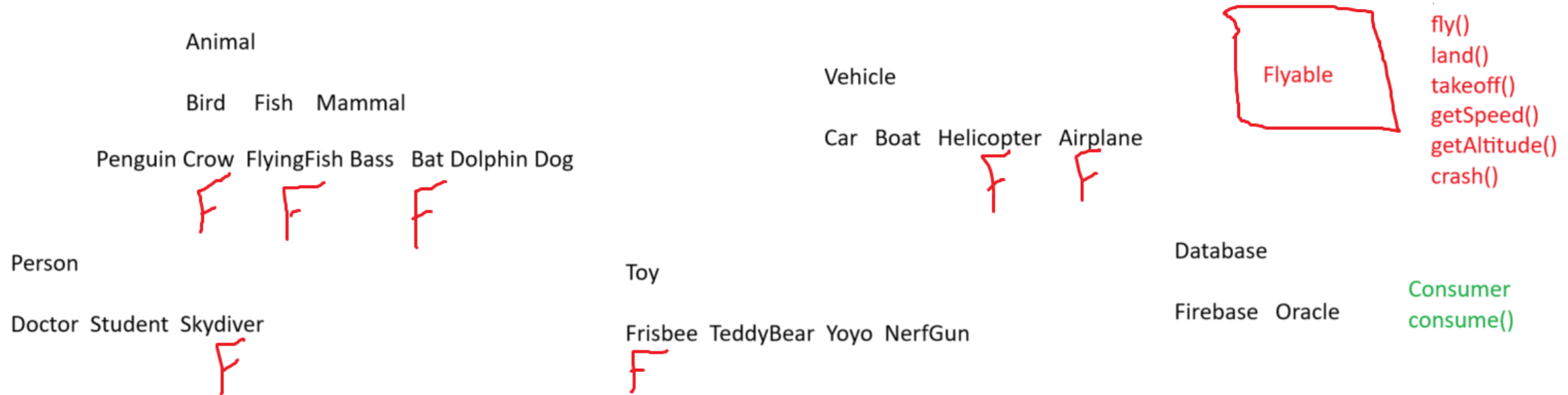
- Functional Interfaces are important
- A functional interface has exactly ONE abstract method
- SAM (single abstract method)
- For lambda expressions
- For method references
- More, in a later lesson
- <https://programming.guide/java/list-of-functional-interfaces.html>

Polymorphism

- Polymorphism is a very important topic for OOP languages
- Java has three types that are involved in polymorphism:

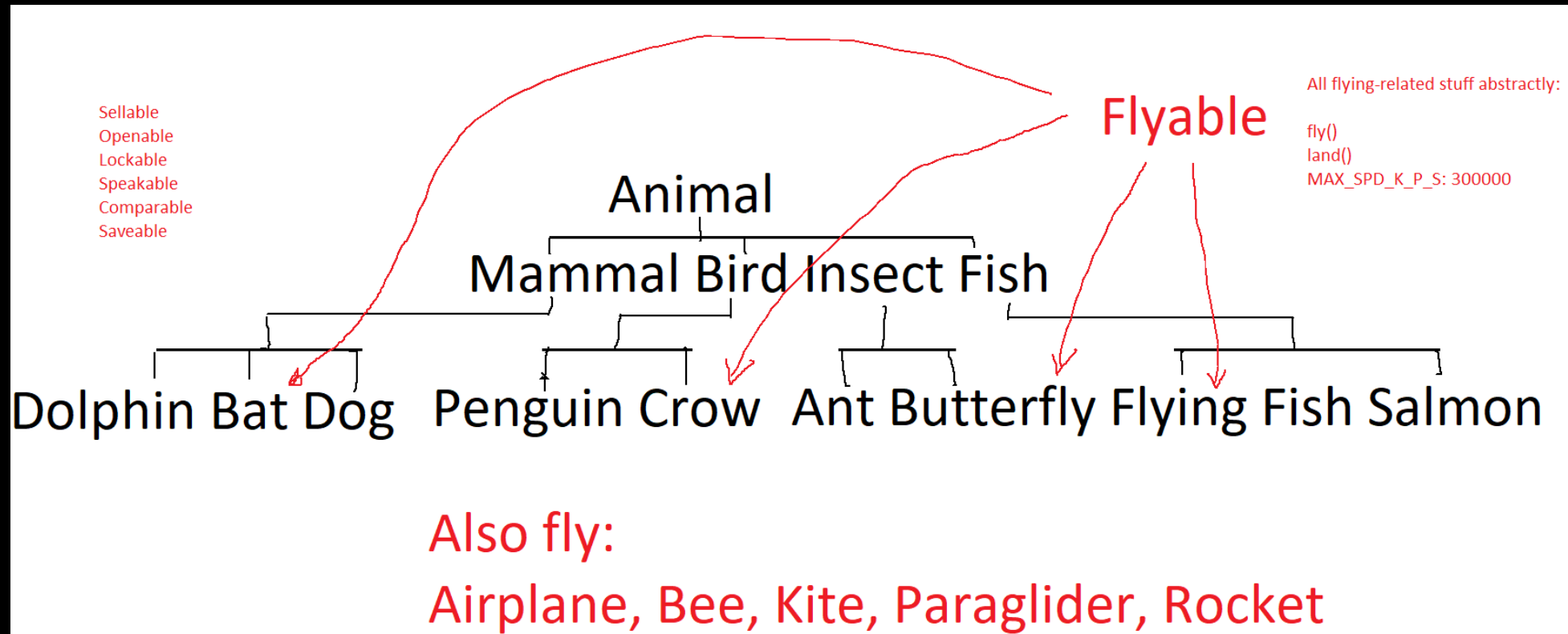
1. Interfaces: if you implement this interface, you must implement its (abstract) methods
2. Abstract classes: no objects can be created of this class, which is still very useful as a parent class containing all common traits for child classes which extend this class
3. Concrete classes: the kind of classes we have used so far: to create objects

Outside of the class hierarchy (1)



An interface is a collection of public static final data and abstract methods (note: now it can also contain "default" concrete methods too)

Outside of the class hierarchy (2)



Inheritance and Interfaces

- An abstract class cannot have objects instantiated of it
- An abstract class can be very useful; it is an *idea*; *put common methods and data there*
- An abstract method has no body: it has only a method signature
- An interface is (in general) a set of abstract methods and public static final data
- An interface can also have default (concrete) methods
- A class can implement zero, one, or more interfaces
- That class must (a) implement all abstract methods and/or (b) be abstract itself

Inheritance: interfaces

```
public interface Speakable
{
    public static final int MAX_DB = 194;
    public abstract void speak();
    void whisper(); // still public and abstract
```

```
    default int getSilenceLevelDb()
    {
        return 0;
    }
}
```

```
class Cat implements Speakable
{
}
```

```
public class Cat implements Speakable
{
    |
}
```

Class 'Cat' must either be declared abstract or implement abstract method 'speak()' in 'Speakable'

Implement methods Alt+Shift+Enter More actions... Alt+Enter

Inheritance: interfaces

```
public interface Speakable
{
    public static final int MAX_DB = 194;
    public abstract void speak();
    void whisper(); // still public and abstract

    default int getSilenceLevel()
    {
        return 0;
    }
}
```

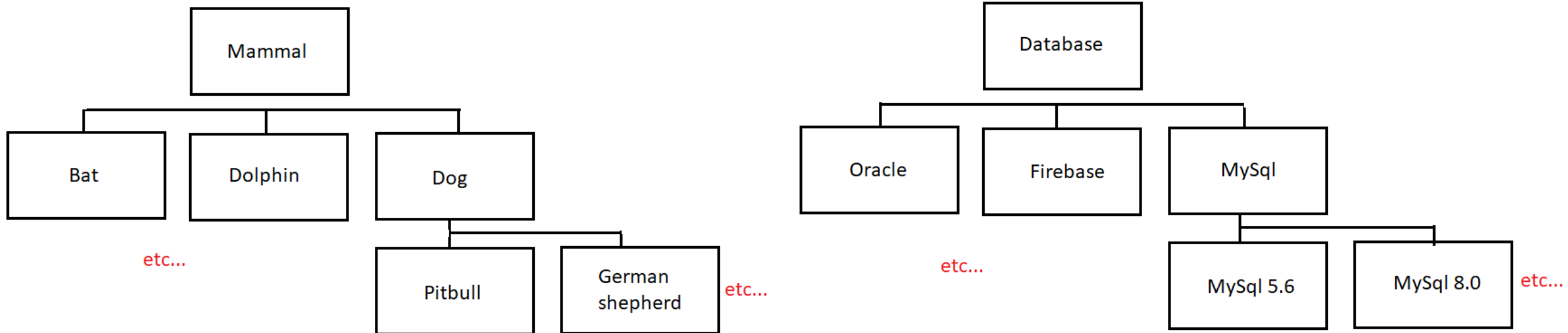
```
public class Cat implements Speakable
{
    @Override
    public void speak()
    {
        System.out.println("MEOW");
    }
    @Override
    public void whisper()
    {
        System.out.println("mew");
    }
    // optionally can Override getSilenceLevel() also
}
```

Inheritance: parent classes

- Any non-final class ("parent class" aka "super class") can be extended by another class ("child class" aka "sub class")
- The parent's non-private data and methods are inherited by the child
- The child's constructors must call one of its parent's constructors with the proper parameters, by calling **super()** in its own constructor (often as its first instruction; see next slide)

Inheritance: Parent Classes

- Many of these classes would be abstract
- Parent classes would have the common methods: speak(), move(), query(), etc....



Inheritance: parent classes

abstract class Animal // abstract: Animal is only an *idea*

```
{  
    private final int yearBorn;  
  
    Animal(final int yearBorn)  
    {  
        this.yearBorn = yearBorn;  
    }  
  
    public int getYearBorn()  
    {  
        return yearBorn;  
    }  
}
```

class Cat extends Animal implements Speakable
{}

```
public class Cat extends Animal implements Speakable
```

```
{
```

There is no default constructor available in 'Animal'

Create constructor matching super Alt+Shift+Enter

More actions... Alt+Enter

```
@Override
```

```
public void speak()
```

```
{
```

```
    System.out.println("MEOW");
```

```
}
```

```
@Override
```

```
public void whisper()
```

```
{
```

```
    System.out.println("mew");
```

```
}
```

```
}
```

Inheritance: Parent Classes

abstract class Animal // abstract: Animal is only an *idea*

```
{
    private final int yearBorn;

    Animal(final int yearBorn)
    {
        this.yearBorn = yearBorn;
    }

    public int getYearBorn()
    {
        return yearBorn;
    }
}
```

```
public class Cat extends Animal implements Speakable
{
```

```
    private final boolean spayedNeutered;
```

```
    public Cat(final int yearBorn, final boolean spayedNeutered)
    {
```

```
        super(yearBorn);
```

```
        this.spayedNeutered = spayedNeutered;
```

```
    }
```

```
    @Override
```

```
    public void speak()
    {
```

```
        System.out.println("MEOW");
```

```
    }
```

```
    @Override
```

```
    public void whisper()
    {
```

```
        System.out.println("mew");
```

```
    }
```

```
    public boolean isSpayedNeutered()
    {
```

```
        return spayedNeutered;
```

```
    }
```

```
}
```

Interface Substitution

- Speakable: interface
- Animal: abstract class, implements Speakable
- Cat: extends Animal

- Cat must therefore also implement Speakable methods (or be abstract itself)

```
import java.util.ArrayList;
import java.util.List;
```

```
class Veterinarian
{
    private List<Animal> animals; // declare as Interface type

    Veterinarian()
    {
        animals = new ArrayList<>(); // ArrayList is a List

        animals.add(new Cat(2020, true)); // Cat is an Animal
        animals.get(0).speak(); // all Animals speak() and whisper()

        // in this example, the Animal will MEOW
    }
}
```

Guaranteed Methods

```
public static void main(final String[] args)
{
    final List<Movable> movables;
    movables = new ArrayList<>();
    movables.add(new Car());
    movables.add(new Dolphin());

    for(final Movable m: movables)    // for-each variables must be final
    {
        m.move();
        m.stop();
    }
}
```

SOLID Principles and Interfaces

- **Single Responsibility Principle (SRP):** Imagine you have a toolbox. Each tool in the box has its own specific job. The hammer hammers nails, the screwdriver tightens screws, and the wrench tightens bolts. Similarly, each interface in Java should focus on doing just one thing, like a tool in your toolbox.
- **Open/Closed Principle (OCP):** Think of a toy building set where you have a bunch of blocks. Once you've built something with those blocks, you shouldn't have to take them apart to add new pieces. Instead, you can just add new pieces on top. Similarly, once you've defined an interface in Java, you shouldn't have to change it every time you want to add a new implementation. You can just create a new class that implements the interface.
- **Liskov Substitution Principle (LSP):** Picture a car and a remote-controlled toy car. You can use the remote to control the toy car just like you'd control a real car with a steering wheel and pedals. In Java, if you have an interface for a car, any class that implements that interface should behave like a car, just as the toy car behaves like a real car when controlled with the remote.
- **Interface Segregation Principle (ISP):** Think of a smartphone. It has multiple features like calling, texting, taking photos, and browsing the internet. But not everyone needs or uses all these features. So, if you were to design interfaces for a smartphone, you'd want to split them up into smaller, specific interfaces like `CallInterface`, `TextInterface`, and `CameraInterface`. This way, each part of your code only needs to depend on the interfaces it actually uses.
- **Dependency Inversion Principle (DIP):** Imagine you're building a house. You have an architect who plans the design (high-level) and construction workers who do the actual building (low-level). Instead of the construction workers directly following the architect's orders, both the architect and construction workers follow a blueprint (abstraction). This way, if you want to change the design, you only need to update the blueprint, and both the architect and construction workers can adjust accordingly. Similarly, in Java, high-level modules (like classes) and low-level modules should both depend on abstractions (like interfaces) rather than directly on each other. This makes your code more flexible and easier to change.

Conflicting Default Methods

```
interface Able{
    void a();
    default void something(){
        System.out.println("aaa something");
    }
}
interface Bable{
    void b();
    default void something(){
        System.out.println("bbb something");
    }
}
class Whatever implements Able, Bable{
    public void a(){
        System.out.println("a");
    }
    @Override
    public void something()
    {
        Able.super.something(); // pick one
    }
    public void b(){
        System.out.println("b");
    }
}
```

Comparable

- <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/Comparable.html>
- Override compareTo
- Return a positive int if “this” is “bigger than” the parameter
- Return a negative int if “this” is “small than” the parameter
- Return 0 if “this” is “equal to” the parameter
- Collections.sort() can use this information

Using compareTo again

```
public class Max
{
    public static void main(final String[] args)
    {
        final Integer[] ints = {5, 44, 321, -6, 0, 31, 34, 300};
        final Character[] chars = {'a', 'g', 'z', 'w', 't'};
        final Double[] doubles = {5.5, 44.44, 321.321, -6.6, 0.0, 31.31, 34.34, 300.3};
        final Dog[] dogs = {new Dog(20, 2000), new Dog(14, 2014), new Dog(17, 2017)};
        final String[] strings = {"hello world", "the end", "how are you", "whatever", "bye"};

        System.out.println(getMax(ints));
        System.out.println(getMax(chars));
        System.out.println(getMax(doubles));
        System.out.println(getMax(dogs));
        System.out.println(getMax(strings));
    }
}
```

```
static <T extends Comparable<T>> T getMax(final T[] a)
{
    T max = a[0];
    for(final T i: a)    {
        if(i.compareTo(max) > 0)    {
            max = i;
        }
    }
    return max;
}

/*
static int getMax(final int[] a)  {
    int max = a[0];
    for(int i: a)    {
        if(i > max)    {
            max = i;
        }
    }
    return max;
}*/
}
```

Inheritance of Interfaces

```
public interface Vehicle {
    void start();
    void stop();
}

public interface Car extends Vehicle {
    void openTrunk();
}

class Sedan implements Car {
    @Override
    public void start() {
        System.out.println("Sedan is starting.");
    }

    @Override
    public void stop() {
        System.out.println("Sedan is stopping.");
    }

    @Override
    public void openTrunk() {
        System.out.println("Sedan's trunk is now open.");
    }
}
```

```
public class Main {
    public static void main(final String[] args) {
        final Car myCar;
        myCar = new Sedan();           // Polymorphism: Car interface reference
        myCar.start();                  // Inherited from Vehicle
        myCar.openTrunk();              // Defined in Car
        myCar.stop();                  // Inherited from Vehicle
    }
}
```